

ALGORITMOS E PENSAMENTO COMPUTACIONAL



Estruturas de repetição:

- Laços de repetição: for, while, do-while
- Controle de fluxo: break, continue, return

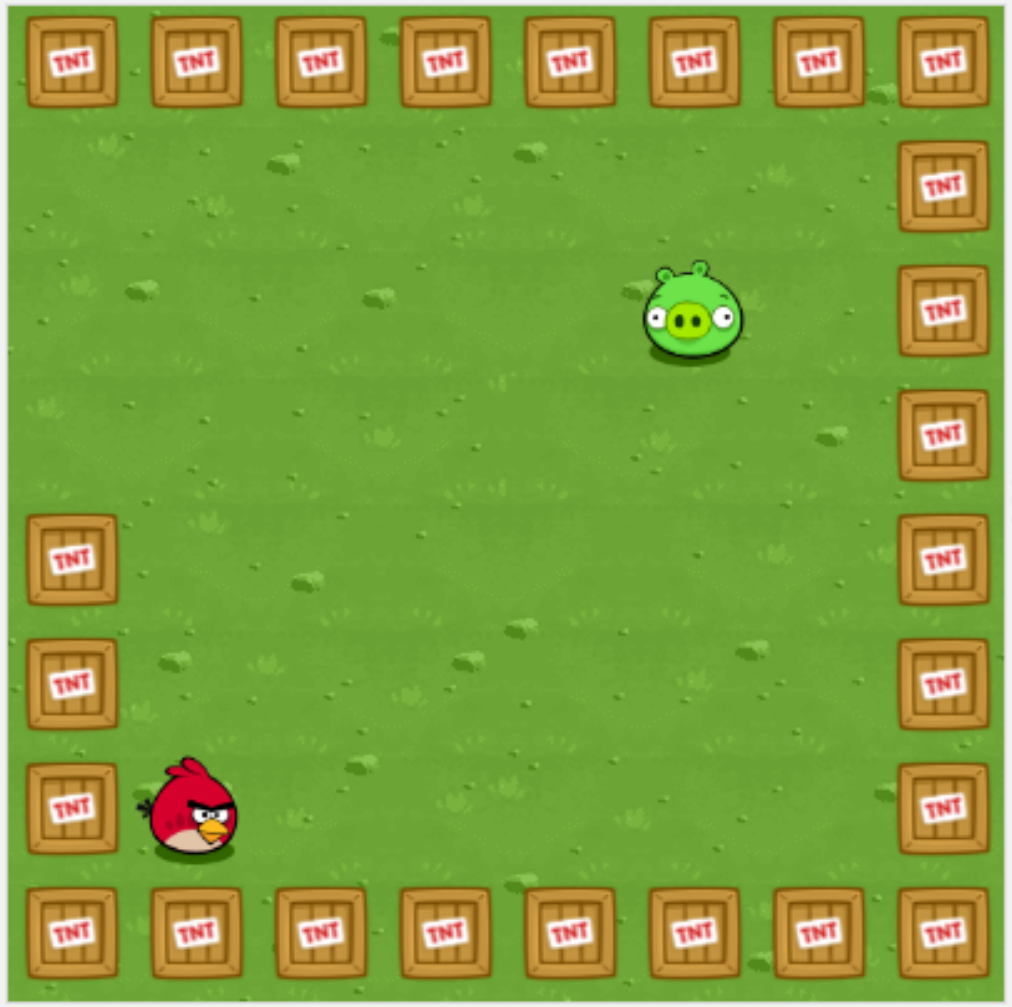
Professor Marco Antonio

2º Semestre 2026

Repetir pra que?????



Pegue o porco da maneira que você puder!



Possível solução

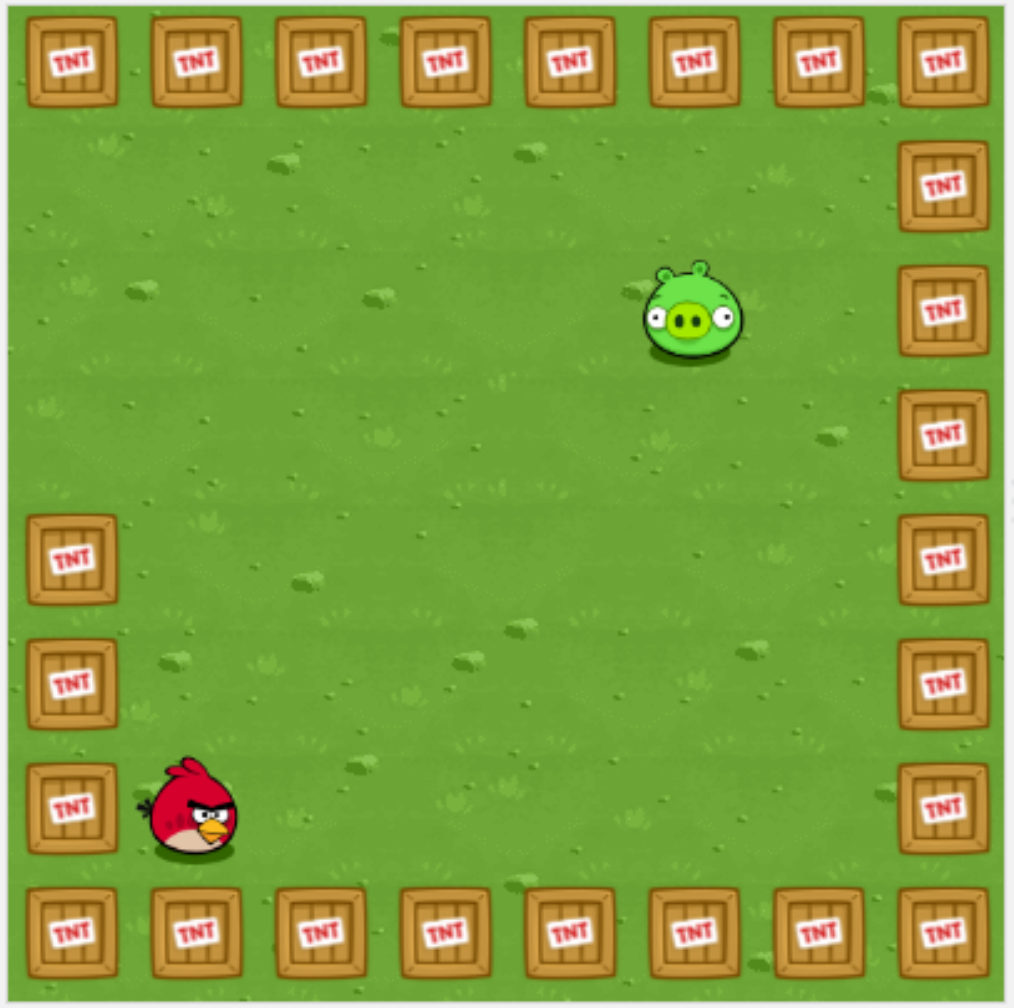


http://go.coderlabs.com.br/angry

Repetir pra que?????

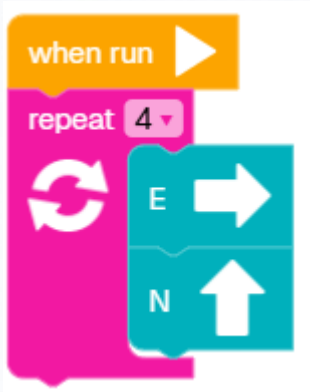


Pegue o porco da maneira que você puder!



ou, talvez essa!!!





http://go.coderlabs.com.br/angry

Conceitos abordados nesta aula

A proposta desta aula é apresentar as estruturas de repetição (looping).

O que são estruturas de repetição?

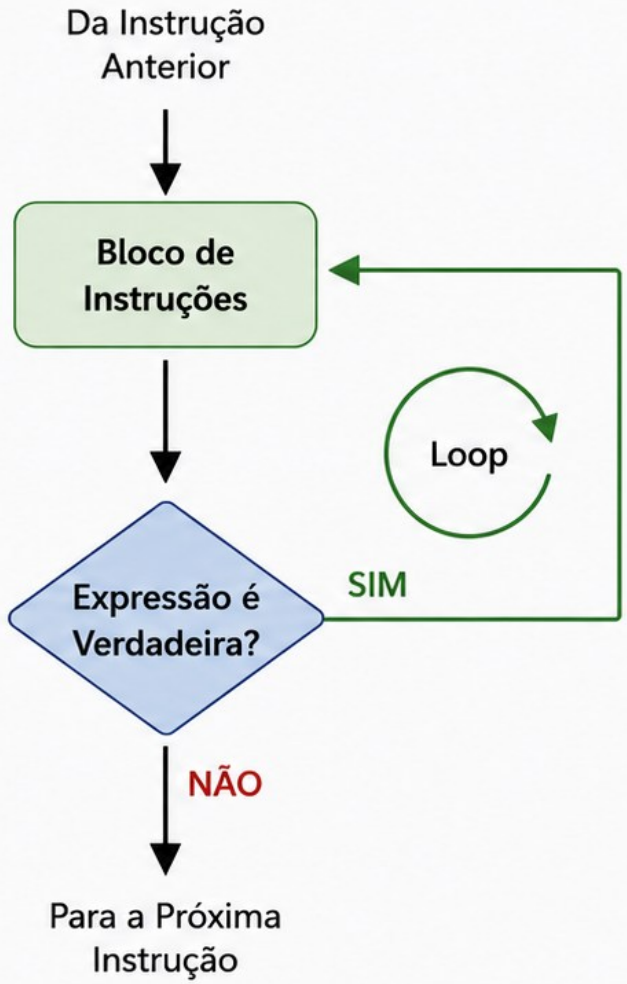
Permitem que um bloco de instruções seja executado **várias vezes**, enquanto uma condição for verdadeira.

- Evita repetir código desnecessariamente
- Torna os programas mais eficientes
- Facilita a solução de problemas repetitivos

Ideia-chave

- Execute o bloco de instruções.
- Verifique a condição.
- → Se for verdadeira, repita tudo.
- → Se for falsa, continue o programa.

Fluxo da estrutura de repetição (loop)



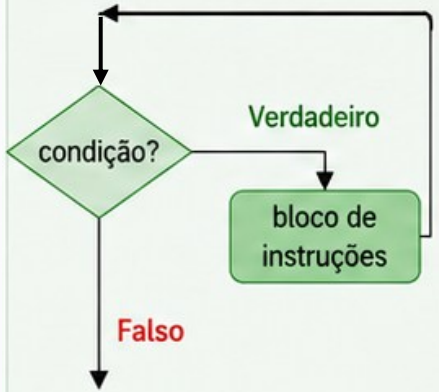
Estruturas de repetição

Permitem executar várias vezes um bloco de instruções **enquanto** determinada condição for **verdadeira**.

As principais estruturas de repetição em pseudocódigo são:

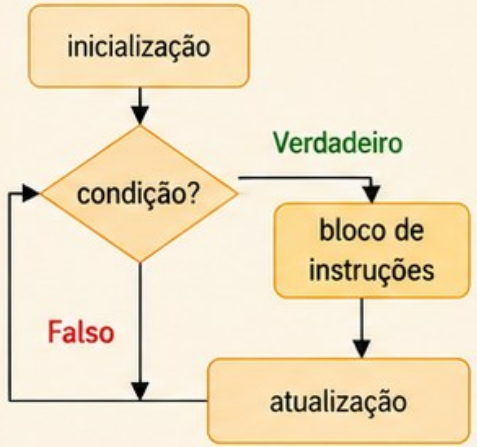
enquanto

Executa enquanto a condição for **verdadeira**.



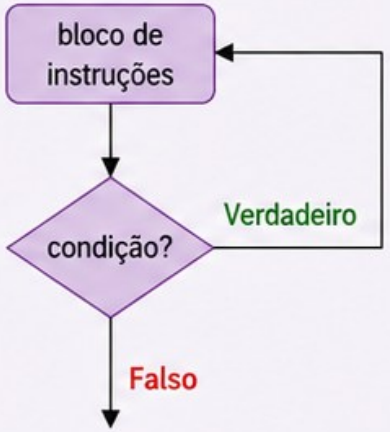
para

Ideal quando sabemos quantas vezes o bloco deve ser executado.



faça... enquanto

Executa o bloco ao menos uma vez e depois verifica a condição.



PseudoCódigo

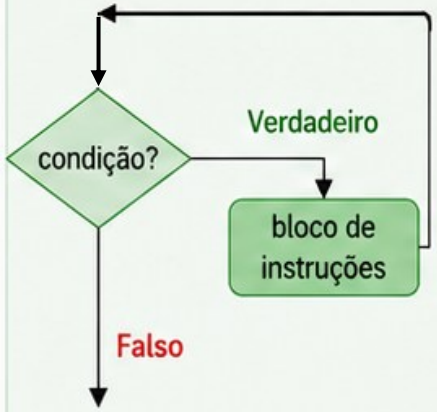
Estruturas de repetição

Permitem executar várias vezes um bloco de instruções **enquanto** determinada condição for **verdadeira**.

As principais estruturas de repetição em C são:

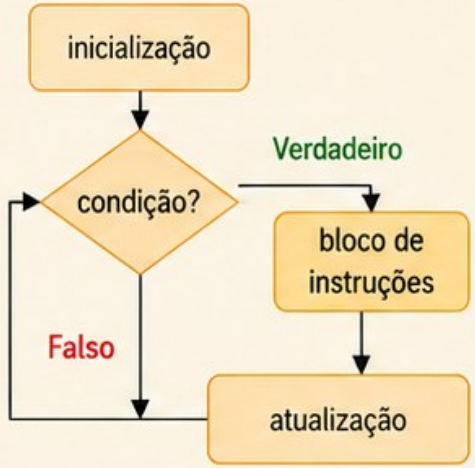
while

Executa enquanto a condição for **verdadeira**.



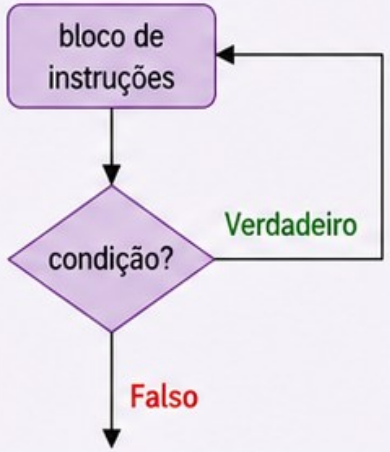
for

Ideal quando sabemos quantas vezes o bloco deve ser executado.



do... while

Executa o bloco ao menos uma vez e depois verifica a condição.



Exemplo em C: soma de 1 até N

```c
#include <stdio.h>

int main() {
    int n, i = 1, soma = 0;
    printf("Digite um número: ");
    scanf("%d", &n);

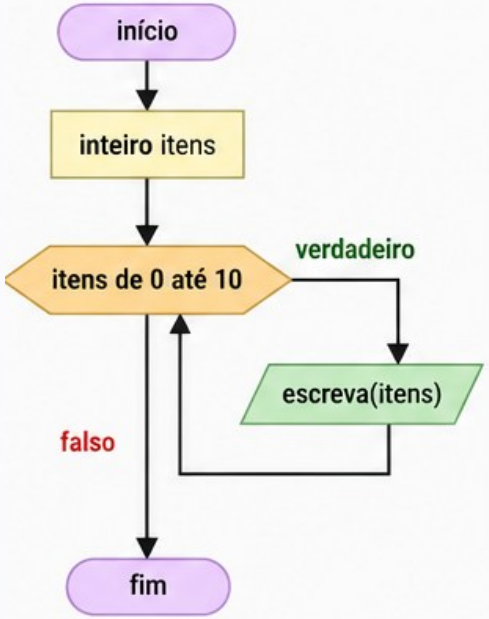
    while (i <= n) {
        soma += i;      // soma = soma + i
        i++;            // i = i + 1
    }

    printf("Soma = %d\n", soma);
    return 0;
}
```

Estruturas de repetição

Laços contados:

Um **contador** irá auxiliar no laço. Neste laço, a repetição da estrutura repete-se até que o contador atinja o limite estipulado na condição.



Exemplo em C

```c
#include <stdio.h>

int main() {
    int itens;

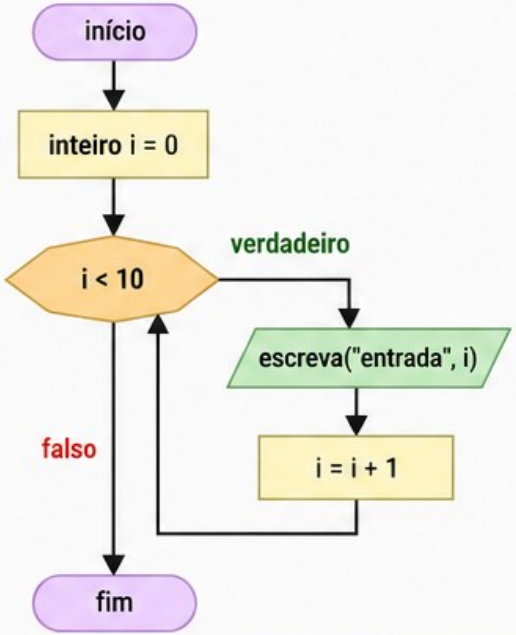
    for (itens = 0; itens <= 10; itens++){
        printf("%d\n", itens);
    }

    return 0;
}
```

Use quando souber exatamente ou tiver um limite definido de repetições.

Laços condicionais:

O valor é desconhecido e devemos utilizar uma variável com valor predefinido em uma condição dentro do laço para finalizarmos a repetição.



Exemplo em C

```c
#include <stdio.h>

int main() {
    int i = 0;

    while (i < 10) {
        printf("entrada %d\n", i);
        i = i + 1;
    }

    return 0;
}
```

Use quando não souber a quantidade de repetições previamente.

Estrutura de repetição



Independente do tipo de laço, todos são constituídos de **três partes**:

1. **Inicialização:** define variável(is) de **controle**.
2. **Condição:** verifica se a repetição deve continuar.
3. **Atualização:** altera variável(is) de **controle**.

while

Executa o bloco enquanto a condição for **verdadeira**.

```c
int i = 1;
while (i <= 10) {
    printf("%d ", i);
    i++;                // atualização
}
```

for

Ideal quando sabemos quantas vezes o bloco deve ser executado.

```c
for (int i = 1; i <= 10; i++) {
    printf("%d ", i);
}
```

do...while

Executa o bloco ao menos **uma vez** e depois verifica a condição.

```c
int i = 1;
do {
    printf("%d ", i);
    i++;                // atualização
} while (i <= 10);
```

Estrutura de repetição para (for)



O laço **for** é compacto, pois a **inicialização**, a **condição** e a **atualização** estão reunidas na declaração do laço.

PSEUDOCÓDIGO

```
para (inicialização; condição; atualização) faça
    // bloco de instruções
fimpara
```

LINGUAGEM C

```
for (inicialização; condição; atualização) {
    // bloco de instruções
}
```



Quando usar?

Quando sabemos exatamente o número de vezes que o bloco deverá ser executado!!!

Estrutura de repetição para (for)

O laço **for** é compacto, pois a **inicialização**, a **condição** e a **atualização** estão reunidas na declaração do laço.

Exemplos C - repeticao.for.alg

```
1 algoritmo repeticao
2     início
3         inteiro i
4         para(i=0; i<10; i++){
5             escreva("Contando ", i, "\n")
6         }
7     fim
```

Inicialização da variável de controle

Atualização

Condição

Quando usar?

Quando sabemos exatamente o número de vezes que o bloco deverá ser executado!!!

Estrutura de repetição para (for)

O laço **for** é compacto, pois a **inicialização**, a **condição** e a **atualização** estão reunidas na declaração do laço.

Exemplos C - repeticao_for.c

```c
#include <stdio.h>

int main(){
    int i;
    for(i=0; i<10; i++){
        printf("Contando %d\n", i);
    }
    return 0;
}
```

Inicialização da variável de controle

Condição

Atualização

Quando usar?

Quando sabemos exatamente o número de vezes que o bloco deverá ser executado!!!

Exemplos de aplicação: algoritmo Tabuada

Escreva um algoritmo que solicite ao usuário um número, calcule e mostre a tabuada desse número.

```
algoritmo Tabuada
    inteiro numero, i
    início
        escreva("Digite um número inteiro: ")
        leia(numero)
        para(i=0; i<=10; i++){
            escreva(numero*i)
        }
    fim
```

Teste de mesa:

Por exemplo: numero = 3



i	Saída
0	0
1	3
2	6
3	9
4	12
5	15
6	18
7	21
8	24
9	27
10	30

Estrutura de repetição enquanto (while)

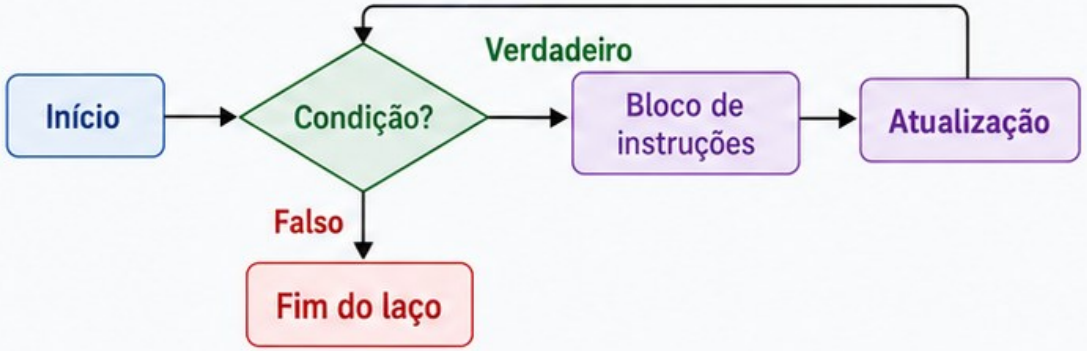
O laço **while** repete um bloco de instruções enquanto uma condição for verdadeira.

PSEUDOCÓDIGO

```
enquanto(condicao){
      //bloco de comandos
}
```

Funcionamento

1. **Condição:** verificada antes de cada repetição.
2. **Bloco:** executado enquanto a condição for verdadeira.
3. **Atualização:** deve permitir que o laço termine.



Atenção!!!
Se a condição nunca for falsa, o loop será **infinito!!!**

Estrutura de repetição enquanto (while)

O laço **while** repete um bloco de instruções enquanto uma condição for verdadeira.

LINGUAGEM C

```
while (condição) {
    // bloco de instruções
}
```

Exemplo em C

```
#include <stdio.h>

int main() {
    int i = 1;

    while (i <= 5) {
        printf("Número: %d\n", i);
        i++;
    }

    return 0;
}
```

Inicialização da variável de controle

Condição

Atualização

Atenção!!!
Se a condição nunca for falsa, o loop será **infinito!!!**

Estrutura de repetição faça… enquanto (do while)

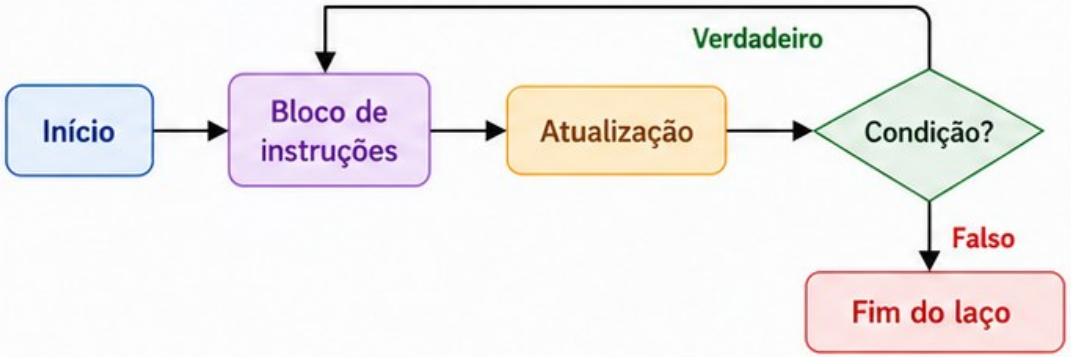
O laço **do...while** executa o bloco de instruções pelo menos uma vez e verifica a condição somente depois.

PSEUDOCÓDIGO

```
faça
    // bloco de instruções
enquanto (condição)
```

Funcionamento

1. **Bloco:** executado antes da primeira verificação.
2. **Condição:** verificada ao final de cada repetição.
3. **Atualização:** deve permitir que o laço termine.



Atenção!!!

No **faça... enquanto**, o bloco é executado pelo menos uma vez, mesmo que a condição seja falsa desde o início.

Estrutura de repetição faça… enquanto (do while)

O laço **do...while** executa o bloco de instruções pelo menos uma vez e verifica a condição somente depois.

LINGUAGEM C

```
do {
    // bloco de instruções
} while (condição);
```

Exemplo em C

```
1  #include <stdio.h>
2
3  int main() {
4      int i = 1;
5
6      do {
7          printf("Número: %d\n", i);
8          i++;
9      } while (i <= 5);
10
11     return 0;
12 }
```

Atualização

Condição

Atenção!!!

No **do... while**, o bloco é executado pelo menos uma vez, mesmo que a condição seja falsa desde o início.

Atenção:
há ponto e vírgula após `while(condição)`

Cheat Sheet: Estruturas de Repetição em C

PRINCIPAIS ESTRUTURAS

while (Pré-teste)

```
while (condicao) {
  fort (condicao) {
    comandos;
  }
}
```

Uso: Quando não se sabe o número de repetições
Regra: Testa a condição antes de executar o bloco
Execuções: `0, 1, 2... n` vezes
Sintaxe: `while (condicao) { comandos; }`

do...while (Pós-teste)

```
do {
  do {
    comandos;
} while (condicao);
}
```

Uso: Quando precisa executar ao menos uma vez
Regra: Testa após executar (atenção ao `;` final)
Execuções: `1, 2, 3... n` vezes
Sintaxe: `do { comandos; } while (condicao);`

for (Determinado)

```
for (int i) {
  for (i <=n 1++) {
    comandos;
  }
}
```

Uso: Quando se sabe previamente a quantidade de iterações
Anatomia: `(inicializacao; condicao; atualizacao)`
Sintaxe: `for (int i = 1; i <= n; i++) { comandos; }`

PADRÕES & ALGORITMOS COMUNS



Contador

Registra quantas vezes um evento ocorreu

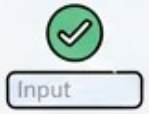
```
if (i % 2 == 0)
  contador++;
```



Acumulador

Armazena soma ou total progressivo

```
soma += valor;
```



Validação

Repete até receber dado válido

```
while (idade < 0
  || idade > 120)
```



Aninhado

Loop dentro de loop (matrizes/grades)

```
for (i...) {
  for (j...) { ... }
```

REGRA DE BOLSO & BOAS PRÁTICAS

- **Regra de Ouro:** Todo loop exige **Inicialização**, **Condição** e **Atualização**
- **Evite Loop Infinito:** Garanta que a condição torne-se falsa em algum momento
- **Laço Aninhado:** O laço interno roda completamente a cada passo do externo

Bulit with MyLens.ai

Hora de praticar!!





Exercícios de aplicação



Em um supermercado, durante o registro de uma compra, o operador do caixa informa sucessivamente os preços dos produtos. Como cada cliente pode adquirir uma quantidade diferente de itens, não é possível determinar previamente quantos preços serão informados.

1- Desenvolva um algoritmo e um programa em C que leia o CPF do cliente e, em seguida, os preços dos produtos adquiridos. A entrada dos produtos deverá continuar até que seja informado o valor 0, indicando o encerramento da compra. Ao final, apresente o CPF do cliente e o valor total da compra.

Caso de teste

```
Entrada:
CPF: 12345678901
Preço: 12.50
Preço: 8.90
Preço: 5.60
Preço: 0

Saída:
CPF: 12345678901
Total da compra: R$ 27.00
```

Exercícios de aplicação



Ao término de uma avaliação, um professor do curso de Ciência da Computação deseja obter alguns indicadores sobre o desempenho de uma turma composta por 5 estudantes.

2- Desenvolva um algoritmo e um programa em C que leia as notas dos 5 estudantes, calcule a média da turma e determine a maior nota obtida.

Caso de teste

```
Entrada:
Digite a nota do 1º estudante: 7.0
Digite a nota do 2º estudante: 8.5
Digite a nota do 3º estudante: 6.0
Digite a nota do 4º estudante: 9.0
Digite a nota do 5º estudante: 5.5

Saída:
Média da turma: 7.20
Maior nota: 9.00
```

Exercícios de aplicação



Durante uma atividade de análise de dados, um sistema recebe uma sequência de números inteiros e precisa considerar apenas os valores ímpares para determinado indicador. A quantidade de valores que será informada não é conhecida previamente.

3- Desenvolva um programa em C que leia números inteiros e some somente os valores ímpares. O número 0 deverá indicar o encerramento da entrada de dados. Ao final, apresente a soma dos números ímpares informados.

Caso de teste

```
Entrada:
Digite um número: 5
Digite um número: 8
Digite um número: 3
Digite um número: 10
Digite um número: 7
Digite um número: 0

Saída:
Soma dos números ímpares: 15
```

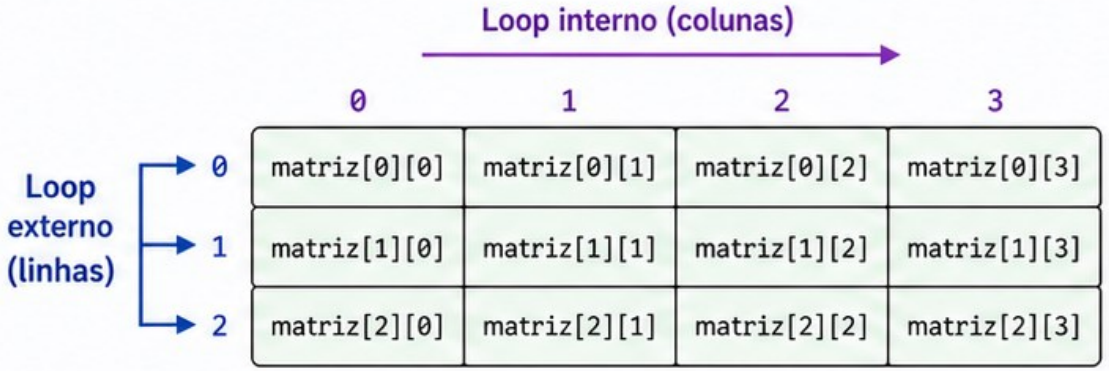
Loops aninhados

Loops aninhados ocorrem quando um **loop está dentro de outro**. São úteis para trabalhar com estruturas complexas, como **matrizes**.

PSEUDOCÓDIGO

```
para(i=0; i<10; i++){
        para(j=0; j<10; j++){
            escreva(matriz[i][j])}
    }
```

1. **Loop externo** (`i`): percorre as linhas da matriz.
2. **Loop interno** (`j`): percorre as colunas da matriz para cada linha.



loop externo: percorre as linhas

loop interno: percorre as colunas

Dica!

Para cada repetição do loop externo, o loop interno executa todas as suas repetições

Loops aninhados

Loops aninhados ocorrem quando um **loop está dentro de outro**. São úteis para trabalhar com estruturas complexas, como **matrizes**.

LINGUAGEM C

```c
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 4; j++) {
        printf("%d ", matriz[i][j]);
    }
    printf("\n");
}
```

Exemplo em C

```c
for (int i = 0; i < 3; i++) {
   for (int j = 0; j < 4; j++) {
      printf("%d ", matriz[i][j]);
   }
   printf("\n");
}
```

Dica!
Para cada repetição do loop externo, o loop interno executa todas as suas repetições

loop externo: percorre as linhas

loop interno: percorre as colunas

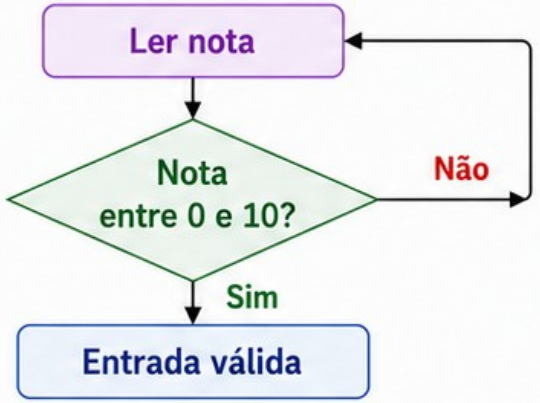
Loops de consistência

Um loop de consistência repete-se até que uma condição de validade seja atendida.
É útil para validar entradas de dados.

PSEUDOCÓDIGO

```
faça
    escreva("Digite a nota do aluno: ")
    leia(nota)
    se (nota < 0 ou nota > 10)
        escreva("Nota inválida!")
enquanto (nota < 0 ou nota > 10)
```

Funcionamento



Atenção!!!

O laço continua enquanto o dado for inválido e termina quando a entrada se torna válida.

Loops de consistência

Um loop de consistência repete-se até que uma condição de validade seja atendida.
É útil para validar entradas de dados.

LINGUAGEM C

```c
#include <stdio.h>

int main() {
    float nota;

    do {
        printf("Digite a nota do aluno: ");
        scanf("%f", &nota);
        if (nota < 0 || nota > 10) {
            printf("Nota inválida!\n");
        }
    } while (nota < 0 || nota > 10);

    printf("Nota válida: %.1f\n", nota);

    return 0;
}
```

Atenção!!!

O laço continua enquanto o dado for inválido e termina quando a entrada se torna válida.

Exercícios de aplicação



Um professor do curso de Análise e Desenvolvimento de Sistemas precisa registrar as notas de sua turma. Para evitar erros de digitação, o sistema acadêmico deve aceitar somente notas pertencentes ao intervalo de 0 a 10.

4- Desenvolva um algoritmo e um programa em C que inicialmente leia a quantidade de alunos da turma. Para cada estudante, solicite sua nota. Caso seja informado um valor menor que 0 ou maior que 10, o programa deverá solicitar uma nova nota e somente prosseguir quando um valor válido for fornecido. Ao final, calcule e apresente a média da turma.

Caso de teste

```
Entrada:
Quantidade de alunos: 3

Nota 1: 8.0
Nota 2: 12.0
Nota inválida!
Digite novamente: 7.0
Nota 3: 6.0

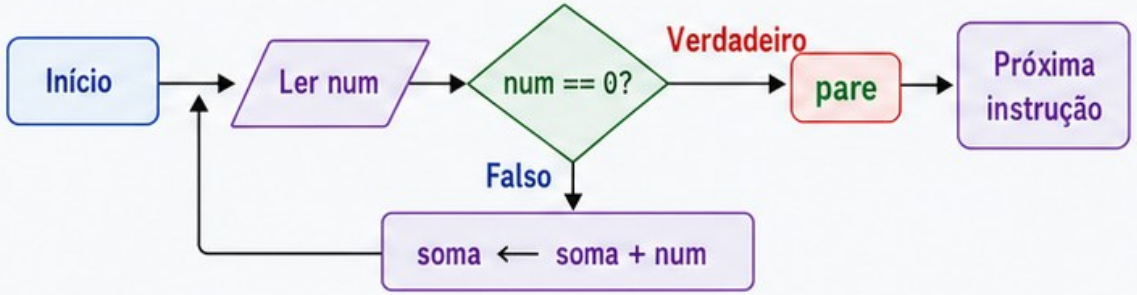
Saída:
Média da turma: 7.00
```

Instruções break e continue

A instrução **pare** (**break**) interrompe o laço, saindo dele e executando a próxima instrução.

PSEUDOCÓDIGO

```
soma ← 0
enquanto (verdadeiro) faça
    escreva("Digite um número: ")
    leia(num)
    se (num == 0) então
        pare
    fimse
    soma ← soma + num
fimenquanto
escreva("Soma = ", soma)
```





Atenção!!!

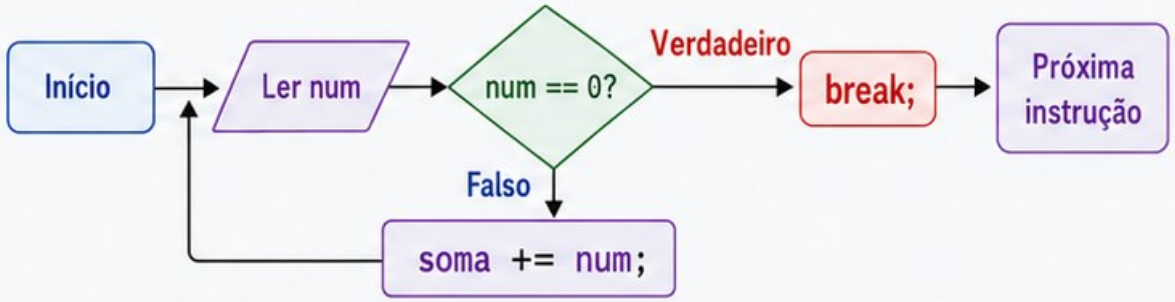
Após o **pare / break**, a execução continua na primeira linha depois do loop.

Instruções break e continue

A instrução pare (break) interrompe o laço, saindo dele e executando a próxima instrução.

LINGUAGEM C

```c
int soma = 0, num;
while (1) {
    printf("Digite um número: ");
    scanf("%d", &num);
    if (num == 0) {
        break;
    }
    soma += num;
}
printf("Soma = %d\n", soma);
```





Atenção!!!

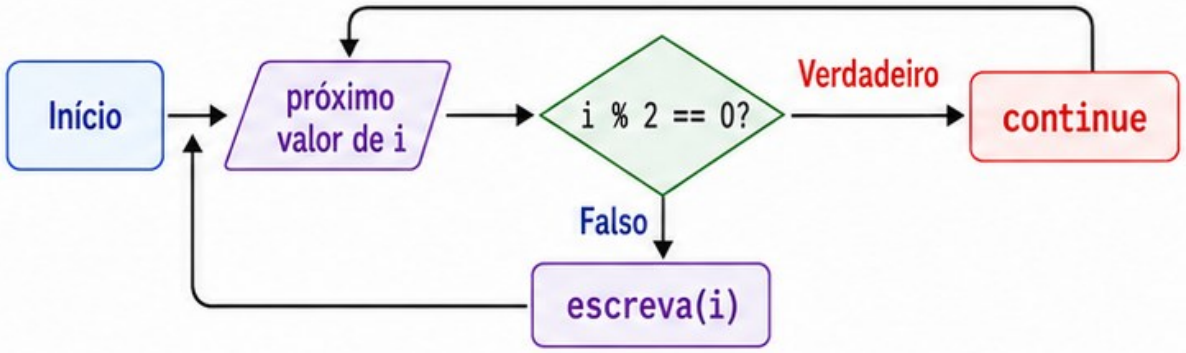
Após o **pare / break**, a execução continua na primeira linha depois do loop.

Instruções break e continue

A instrução **continue** interrompe o ciclo atual de um laço, passando para o próximo ciclo do mesmo laço.

PSEUDOCÓDIGO

```
para (i ← 0; i <= 10; i ← i + 1) faça
    se (i % 2 == 0) então
        continue
    fimse
    escreva(i)
fimpara
```



Atenção!!!

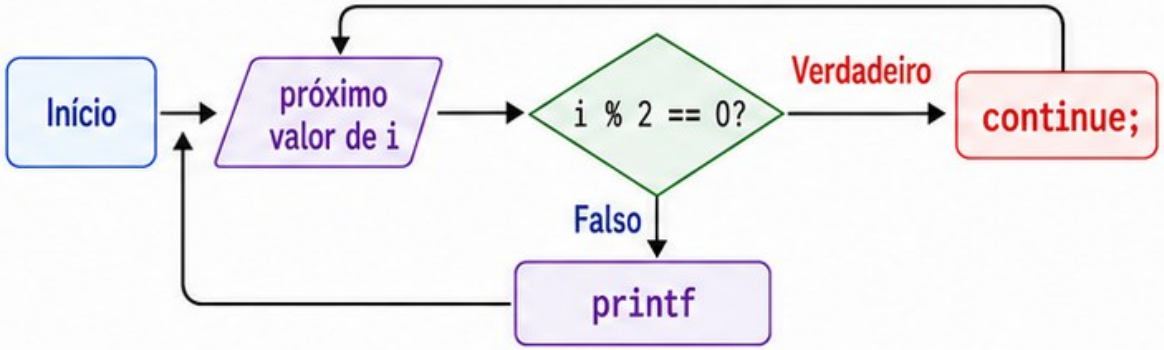
O **continue** não encerra o loop; apensa ignora o restante da iteração atual.

Instruções break e continue

A instrução **continue** interrompe o ciclo atual de um laço, passando para o próximo ciclo do mesmo laço.

LINGUAGEM C

```
for (int i = 0; i <= 10; i++) {
  if (i % 2 == 0) {
     continue;
  }
  printf("%d ", i);
}
```



Atenção!!!

O **continue** não encerra o loop; apensa ignora o restante da iteração atual.

Exercícios de aplicação

Como medida básica de segurança, um sistema bancário limita a quantidade de tentativas consecutivas de autenticação. Após três tentativas incorretas, o acesso deve ser bloqueado.

5- Desenvolva um algoritmo e um programa em C que solicite o número da conta e a senha do cliente. Considere como credenciais válidas a conta 12345 e a senha 123. O usuário poderá realizar no máximo três tentativas. O programa deverá encerrar imediatamente quando as credenciais corretas forem fornecidas ou bloquear o acesso após a terceira tentativa incorreta. Utilize `do...while`.

Caso de teste

```
Entrada:
Conta: 12345
Senha: 111
Conta: 12345
Senha: 222
Conta: 12345
Senha: 123

Saída:
Acesso autorizado.
```

Exercícios de aplicação



Um restaurante deseja disponibilizar um terminal de autoatendimento no qual o cliente possa incluir sucessivamente produtos em seu pedido. O sistema possui o seguinte cardápio a seguir.

6- Desenvolva um algoritmo e um programa em C que apresente o cardápio e permita ao cliente adicionar produtos informando seus códigos. Após cada produto válido, apresente o subtotal do pedido. O código 0 deverá finalizar a compra e apresentar o valor total. Utilize do...while.

```
Cafeteria Anália

[1] X-Burger (R$ 18,90)
[2] Batata frita (R$ 9,50)
[3] Suco (R$ 10,00)
[4] Pudim (R$ 12,00)
[5] Sair

Digite uma opção:
```

Exercícios de aplicação

Um restaurante deseja disponibilizar um terminal de autoatendimento no qual o cliente possa incluir sucessivamente produtos em seu pedido. O sistema possui o seguinte cardápio a seguir.

```
Cafeteria Anália

[1] X-Burger (R$ 18,90)
[2] Batata frita (R$ 9,50)
[3] Suco (R$ 10,00)
[4] Pudim (R$ 12,00)
[5] Sair

Digite uma opção:
```

Caso de teste

Entrada:

```
Digite uma opção: 1
Digite uma opção: 3
Digite uma opção: 2
Digite uma opção: 0
```

Saída:

```
Subtotal: R$ 18.00
Subtotal: R$ 25.00
Subtotal: R$ 37.00
Total do pedido: R$ 37.00
```

Alguma dúvida????



Universidade Cruzeiro do Sul

CRÉDITOS

Material elaborado por:

Prof. Marco Antonio Sanches

Material base elaborado por:

- Prof. Alcides
- Prof. Lédon
- Prof. Ana Paula
- Profa. Cristiane

Última atualização: 2025/2

CONTATOS



marco.sanches@cruzeirodosul.edu.br



marco-antonio-sanches-anastácio-601b9123